

WebAssembly와 컨테이너의 실환경 비교를 통한 IoT 로그 파이프라인 계층별 런타임 할당

Layer-Specific Runtime Allocation in IoT Log Pipelines through an Empirical Comparison of WebAssembly and Containers

2021112053 고동현 동국대학교 컴퓨터공학과

요약. WebAssembly(Wasm) 서버리스 런타임인 SpinKube는 컨테이너 대비 콜드 스타트(cold-start)·밀도·이미지 크기에서 수백 배의 우위를 주장하나, 이러한 수치는 격리된 마이크로벤치마크에서 도출된 것으로 쿠버네티스 운영 환경에서의 중단 간 검증은 부족하다. 본 연구에서는 IoT 로그 파이프라인을 네 계층(Edge, Transform, Ingestion, Storage)으로 모델링하고, 그중 Edge·Transform·Ingestion 세 계층을 동일한 Go 워크로드의 SpinKube Wasm 모듈과 컨테이너로 구현하여 AKS 위에서 실측 비교하였다(Storage는 스토리지 엔진의 영향이 지배적이어서 측정에서 제외). 측정 결과 런타임 수준의 정량 우위는 쿠버네티스 운영 스택을 통과하며 상당 부분 희석되었고, 계층마다 상반된 양상이 나타났다. 고처리량이 요구되는 Ingestion 계층에서는 컨테이너가 우세한 반면, 테넌트별(per-tenant) 격리가 요구되는 Transform 계층에서는 Wasm 공유 풀이 더 높은 테넌트 밀도와 더 낮은 테넌트당 비용으로 우세하였다. 이에 본 연구는 Ingestion 계층은 컨테이너에, Transform 및 Edge 계층은 Wasm에 할당하는 계층별 런타임 배치를 제안한다.

키워드: WebAssembly, SpinKube, 쿠버네티스, 서버리스, IoT, 멀티테넌시

1 서론

대규모 IoT 환경에서 발생하는 로그의 처리는 세 가지 요구를 동시에 안고 있다. 다수의 디바이스가 동시에 신호를 보내면서 발생하는 스파이크 트래픽을 흡수해야 하고, 테넌트마다 서로 다른 커스텀 필터 규칙을 격리된 상태로 적용해야 하며, 저사양 엣지 장치부터 클라우드 게이트웨이까지 이종 배포 환경에 동일한 처리 로직을 이식할 수 있어야 한다. 더 나아가 IoT 로그 파이프라인은 단일 컴포넌트가 아니라 Edge, Transform, Ingestion, Storage로 이어지는 다층 구조이며, 각 계층이 요구하는 처리량, 격리, 이식성의 우선순위가 서로 다르다. 따라서 어느 한 런타임이 파이프라인 전체에 일률적으로 최적이라고 보기는 어렵다. 다만 Storage 계층은 런타임 선택보다 데이터 베이스 및 스토리지 엔진의 영향이 지배적이므로, 본 연구의 계층별 분석은 Edge, Transform, Ingestion 세 계층에 집중한다.

이러한 배경에서 WebAssembly(Wasm) 기반 서버리스 런타임, 특히 SpinKube와 containerd-shim-spin이 대안으로 제시되어 왔다. 관련 자료들은 Wasm이 컨테이너 대비 cold-start 약 400×, Pod 밀도 약 43×, 이미지 크기 약 8×의 우위를 가진다고 주장한다. 그러나 이 수치들은 대부분 격리된 Wasmtime 마이크로벤치마크에서 도출된 것으로, 쿠버네티스라는 실제 운영 스택을 통과한 뒤 end-to-end 경로에서 관찰되는 값은 충분히 보고되지 않았다. 즉, 런타임 수준의 이론적 우위가 운영 환경의 오케스트레이션, shim, SDK 계층을 거치며 얼마나 유지되는지에 관한 검증 공백이 존재한다.

본 연구에서는 이 공백을 메우기 위하여, IoT 로그 파이프라인을 네 계층으로 모델링하고 그중 Edge·Transform·Ingestion 세 계층을, 동일한 Go 워크로드의 SpinKube Wasm 모듈과 scratch 컨테이너로 각각 구현하여 Azure Kubernetes Service(AKS 1.34) 위에서 실측 비교한다. 언어와 비즈니스 로직, 부하 조건을 통제하여 순수한 런타임 차이만을 격리 측정하였으며, 다수의 측정 세션에 걸쳐 실측 비교를 수행하였다. 본 연구가 답하고자 하는 질문은 네 가지로, 고동시성 수집 계층에서 두 런타임 중 어느 쪽이 우위인지, 멀티테넌트 per-tenant 격리가 요구될 때 Wasm의 밀도 이점이 실환경에서 어느 정도 실현되는지, 마케팅이 주장한 정량 이점이 각 계층에서 얼마나 희석되는지, 그리고 ”

인스턴스화(instantiation) 0.5ms” 주장이 격리된 Wasmtime에서 성립하는지를 다룬다.

측정 결과 세 가지 대표적 양상이 나타났다. 첫째, instantiation 실측값은 0.436ms로 벤더 주장과 통계적으로 구별되지 않았으나($p=0.71$), 전체 요청 경로의 72%가 쿠버네티스, shim, SDK, HTTP 계층에서 더해져 런타임 수준의 우위는 운영 경로에서 희석되었다. 둘째, Ingestion 계층에서는 컨테이너가 처리량에서 유의하게 우세하였으나(약 19–22×, 24–28K vs 1,264 rps), Transform 계층에서 per-tenant 격리를 강제하면 반대 양상으로 나타나, 컨테이너 1:1 격리가 약 41 Pod에서 한계에 이르는 반면 고정 Wasm 풀은 8,000 테넌트를 무손실로 수용하며 테넌트당 비용은 최대 64× 저렴하였다. 셋째, 영점 스케일링(scale-to-zero) 콜드 스타트와 운영 자동복구에서는 Wasm이 우위였으나 버전 롤백과 설정 교체에서는 컨테이너가 8–10× 빨랐다. 본 연구는 이러한 실측에 근거하여, Ingestion 계층은 컨테이너에, Transform과 Edge 계층은 Wasm에 할당하는 계층별 런타임 할당 방안을 제안한다.

2 관련 연구

WebAssembly(Wasm) 기반 서버리스 성능에 관한 선행 연구는 크게 단일 런타임 마이크로벤치마크와 시스템 수준 설계로 구분된다. Faasm[1]은 SFI(Software Fault Isolation) 기반의 경량 격리 추상화를 제안하여 컨테이너 대비 약 2배의 속도와 10배의 메모리 절감을 보고하였으나, 쿠버네티스 오케스트레이션 환경에서의 동작은 연구 범위 밖이었다. Lumos[2]는 엣지-클라우드 연속체에서 Wasm 런타임을 정량 분석하여 AoT(Ahead-of-Time) 컴파일 시 이미지 크기 약 30배 감소와 cold-start 약 16% 감소를 제시하였다. Groundhog[3]은 Wasm SFI를 사용하지 않고 컨테이너 재사용에 기반한 FaaS 요청 격리를 다루어, 격리 기법의 일반론에서 본 연구와 상보적 위치를 차지한다. 한편 Wiegratz[4]는 독립적으로 쿠버네티스 환경의 Wasm과 컨테이너를 비교하여 “Wasm은 startup 오버헤드가 존재하나 long-running 워크로드에서는 무시할 수준이며, 격리 측면에서 우세하나 만능 해법은 아니다”라고 결론하였고, 이는 본 연구의 실측 결과와 정합한다.

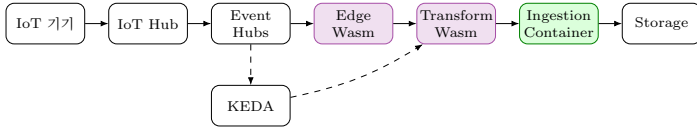


그림 1: 제안하는 IoT 로그 파이프라인 구조 (보라: Wasm, 초록: Container)

산업계에서는 Wasm이 이미 Transform 계층에서 확장 메커니즘으로 채택되고 있다. Envoy[6]는 HTTP 필터 확장에 Wasm을 도입하였으며(다만 공식 문서에서 실험적 단계로 표기됨), Fluent Bit[7]는 로그 처리 파이프라인의 필터 단계에 성숙한 Wasm 플러그인 인터페이스를 제공한다. 이는 본 연구가 Transform 계층에 Wasm을 권장한 결과와 부합한다. 그러나 이상의 어느 연구도 쿠버네티스 운영 수준에서 벤더가 주장한 정량 이점을 파이프라인의 계층별로 분해하여 검증하지는 않았다. 본 연구에서는 동일한 Go 워크로드를 SpinKube Wasm 모듈과 컨테이너로 구현하고 AKS 위에서 Edge·Transform·Ingestion 세 계층에 걸쳐 실측함으로써, 마케팅 수치가 운영 스택을 통과하며 어느 계층에서 유효하고 어느 계층에서 희석되는지를 정량적으로 규명한다는 점에서 선행 연구와 구별된다.

3 제안 방법 및 실험 환경

3.1 4계층 IoT 로그 파이프라인 모델

본 연구에서는 IoT 로그 처리를 단일 컴포넌트가 아니라 서로 다른 요구사항을 가진 다층 구조로 보고, 이를 Edge·Transform·Ingestion·Storage의 네 계층으로 모델링한 IoT 로그 파이프라인을 제안한다. Edge 계층은 저사양 엣지부터 클라우드 게이트웨이까지 이중 환경에 배포되어 로그를 수집하므로 빠른 기동과 이식성이 요구된다. Transform 계층은 테넌트별 커스텀 필터 규칙을 적용하므로 테넌트 단위의 격리와 밀도가 핵심 요구사항이 된다. Ingestion 계층은 대규모 동시 디바이스에서 발생하는 스파이크 트래픽을 수집하므로 높은 처리량이 관건이다. Storage 계층은 런타임 선택보다 데이터베이스 및 스토리지 엔진의 영향이 지배적이고 런타임에 따른 차이가 상대적으로 작으므로 분석 대상에서 제외하였다. 전체 시스템 구조는 그림 1에 나타내었다.

본 연구의 핵심 가설은 계층별 요구사항이 상이하므로 단일 런타임을 전 계층에 일괄 적용하기보다 계층별로 런타임을 할당하는 편이 유리하다는 것이다. 구체적으로, 빠른 기동과 이식성이 요구되는 Edge 계층과 테넌트 격리가 요구되는 Transform 계층에는 WebAssembly(SpinKube) 런타임을, 고 처리량이 요구되는 Ingestion 계층에는 컨테이너 런타임을 할당하는 방안을 제안한다. 다만 본 연구는 네 계층을 단일 파이프라인으로 직렬 연결하여 측정하는 것이 아니라, 각 계층의 요구사항을 개별 실험으로 측정하여 권장 배치를 도출하였다.

3.2 통제변수와 실험 설계

런타임 차이만을 격리 측정하기 위해 본 연구에서는 독립 변수를 런타임(SpinKube Wasm 모듈 대 scratch 컨테이너)으로 한정하고 나머지 조건을 모두 통제하였다. 두 구현은 동일한 Go 언어로 작성되었고, JSON 파싱과 임계값 분석이라는 동일한 비즈니스 로직을 수행하며, 동일한 쿠버네티스 환경과 동일한 부하 조건에서 측정되었다. 경량 워크로드

를 의도적으로 선택한 까닭은 복잡한 로직이 개입하면 언어나 알고리즘 차이가 섞여 순수한 런타임 차이를 분리할 수 없기 때문이다. 리소스 요청은 각 런타임의 문서 권장 최소값을 따라 Wasm은 16 Mi/50 m, 컨테이너는 128 Mi/100 m로 설정하였다. Spin은 Go 로직을 wasip1으로 컴파일하고 containerd-shim-spin이 모듈을 containerd task로 실행하며 spin-operator가 SpinApp CRD를 Deployment로 reconcile하고, 컨테이너는 동일 로직의 Go scratch 이미지로 구성하였다.

3.3 Wasm 구현상의 고려사항

Spin의 wasip1 실행 모델에서 Go 모듈은 요청 단위(per-request)로 인스턴스화되어 처리 직후 폐기된다. 본 연구에서는 TinyGo의 conservative 가비지 컬렉션(GC)을 사용하였다(초기에 시도한 leaking GC는 wit-component 어댑터와의 호환 문제로 폐기하였다). 요청 단위로 인스턴스가 종료되므로 전통적인 장기 실행 프로세스에서 누적되는 메모리 누수의 영향은 제한적이며, 따라서 후술하는 메모리·지연 지표는 런타임의 본질적 특성을 반영한 것이지 누수나 로직 결함에 기인한 것이 아니다.

3.4 실험 환경

실험은 Azure Kubernetes Service(AKS 1.34) 위에서 수행하였다. 노드는 Standard.B2s.v2 3대로 총 6 vCPU를 구성하였다. IoT 종단 경로를 위해 Azure IoT Hub(B1)와 Event Hubs(Kafka API), 이미지 레지스트리로 ACR을 사용하였으며, 클러스터에는 spin-operator, containerd-shim-spin v0.17.0, KEDA, cert-manager를 설치하였다. KEDA는 Event Hubs 큐 길이를 감지하여 Pod를 자동 증설한다. 측정 경로는 두 가지로, LoadBalancer를 통한 직접 HTTP 경로와 IoT Hub에서 Event Hubs를 거치는 MQTT 종단 경로를 함께 사용하였다.

3.5 측정 방법

측정값의 정규성은 Shapiro-Wilk 검정으로 확인한 뒤 Welch's t 검정 또는 Mann-Whitney U 검정을 자동 선택하여 적용하였다. confirmatory 10개 지표 family에 대해서는 Bonferroni 보정을 적용하였으며($\alpha_{adj} = 0.005$), 효과 크기는 Cohen's d로 산출하였다. 소표본이거나 단일 실행으로 얻은 결과는 descriptive로 구분하여 보고하였다. 또한 instantiation 시간은 kubectl logs에 노출되지 않고 shim journal에만 기록되므로, 특권 DaemonSet에서 nsenter와 journalctl을 통해 추출하였다. 외부 LoadBalancer를 경유하는 부하 생성은 호스트와 외부 LoadBalancer 사이의 네트워크 병목을 유발하므로, 클러스터 내부 부하 생성 Pod를 사용하여 측정하였다. 본 연구에서 사용하는 지연 지표는 측정 구간으로 구분된다. instantiation은 Wasmtime이 모듈을 인스턴스화하는 시간(shim journal 기준), Pod Ready는 Pod 생성부터 Ready 도달까지, 이미지 Cold Pull은 노드에 이미지가 없는 상태에서 이미지 fetch 완료까지, scale-from-zero cold-start는 replicas 0→1 전환부터 첫 응답(TTFR)까지의 시간을 의미한다.

표 1: 주요 실험 결과 요약 (측정 지표 $n = 10$)

지표	Wasm	Container	p	우세
Instantiation	0.436 ms	0.5 ms ^a	0.71	비유의 ^c
Cold Start	269 ms	2,979 ms	<0.005	Wasm
Pod Ready	1,085 ms	2,161 ms	<0.01	Wasm
Throughput	1,264 rps	24–28K rps	<0.005	Container
Tenant Density	8,000	41	– ^b	Wasm
Cost/Tenant	\$0.07	\$4.69	– ^b	Wasm

^a Fermion 주장값(이론, 미측정). ^b 결정론적 모델값(표본분포가 없어 p 미적용). ^c 차이를 입증하지 못함(검정력 제한, 동등성 미검정). Cost/Tenant는 2,500 테넌트 기준.

4 실험 및 결과

본 연구에서는 동일한 Go 워크로드를 SpinKube 기반 WebAssembly(이하 Wasm) 모듈과 scratch 컨테이너로 구현하여 Azure Kubernetes Service(AKS 1.34) 위에서 IoT 로그 파이프라인 네 계층 중 Edge·Transform·Ingestion 세 계층에 대해 측정하였다. 독립변수는 런타임으로 한정하고, 언어와 비즈니스 로직, 쿠버네티스 환경, 부하 조건을 통제하였다. 통계 검정은 Shapiro-Wilk 정규성 검정 후 Welch의 t 검정 또는 Mann-Whitney U 검정을 자동 선택하였으며, 확정적(confirmatory) 10개 지표군에 대해 Bonferroni 보정($\alpha_{adj} = 0.005$)을 적용하였다. 주요 측정 결과를 먼저 표 1에 요약한다.

4.1 런타임 순수 성능과 운영 계층 희석

Wasmtime instantiation 시간을 containerd journal 파싱으로 직접 측정한 결과 0.436 ms($n=10$)로, Fermion[5]이 주장한 0.5ms와 통계적으로 구별되지 않았다($p=0.71$). 즉 런타임 수준의 정량 주장은 기각되지 않았다(단 $n = 10$ 으로 검정력이 제한되므로, 이는 동등성의 증거가 아니라 차이를 입증하지 못한 것으로 해석해야 한다). 그러나 전체 요청 경로 1.55 ms 가운데 런타임이 차지하는 비중은 0.44 ms(28%)에 불과하였고, 나머지 72%(1.11 ms)가 쿠버네티스, shim, SDK, HTTP 등 운영 계층에서 더해졌다. 따라서 마케팅이 제시한 런타임 수준의 우위는 실제 운영 스택을 통과하면서 상당 부분 희석되는 것으로 나타났다. 다만 1.11 ms의 계층별 세부 귀속(shim, spin-SDK, 네트워킹)은 각 계층에 독립 probe를 두지 않아 분해하지 못하였으며, 본 연구는 이를 집계 오버헤드로만 보고하고 세분은 future work로 남긴다.

4.2 Ingestion 계층의 처리량 비교

고동시성 로그 수집 계층에서는 컨테이너가 우세하였다. 클러스터 내부 부하 생성기(fortio)를 사용한 spike 측정에서 컨테이너는 25K부터 100K target에 이르는 전 구간에서 24–28K rps를 일관되게 유지한 반면, Wasm은 5-Pod 풀 기준 약 1,264 rps에서 한계에 이르렀다(그림 2). 초기 외부 LoadBalancer 경유 측정에서 관찰된 고부하 구간의 반대 양상은 호스트와 외부 LB 사이 네트워크 아티팩트로 판명되어 클러스터 내부 재측정으로 기각하였으며, 이는 부하 생성 위치가 결과를 좌우할 수 있다는 방법론적 교훈으로 정리된다.

§5.12 In-cluster spike: Container sustains 24–28K rps, Wasm plateaus at ~1,264 rps (v5 'reversal' was a host-LB artifact)

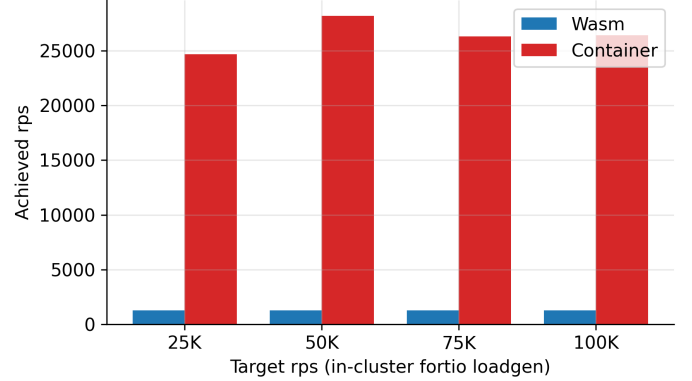


그림 2: In-cluster spike 처리량 — 컨테이너 24–28K rps, Wasm 약 1,264 rps 한계.

4.3 Transform 계층의 per-tenant 격리

테넌트당 1 Pod로 격리를 강제하는 Transform 계층에서는 반대 양상으로 나타났다. 컨테이너의 메모리 요청(128 Mi/Pod)이 노드의 할당 가능량(Node Allocatable)을 소진하면서 추가 Pod가 스케줄링되지 못하고 Pending 상태에 머물러, 컨테이너 1:1 격리는 약 41 Pod에서 한계에 이르렀다. 이 한계는 OOM(Out-of-Memory) kill이 아니라, 쿠버네티스 스케줄러가 resources.requests 예약량을 기준으로 노드 용량을 산정하기 때문에 발생한 것이다. 실측에서 컨테이너의 실제 RSS는 Pod당 2 Mi로 작았으나, 한계를 지배한 것은 실사용량이 아니라 요청 예약량(128 Mi×Pod 수)이었다. 반면 고정 Wasm 공유 풀은 8,000 테넌트(1,600 rps)까지 무손실로 수용하였으며, p99 지연만 255 ms에서 3,720 ms로 단조 증가하였다(그림 3). 수용 한계는 처리량 벽이 아니라 지연 SLO의 함수로, p99 < 500 ms 기준 약 1,000 테넌트, p99 < 1s 기준 약 2,500 테넌트, 무손실 기준 8,000 테넌트 이상으로 나타났다. 각 테넌트가 0.2 rps를 발생시킨다는 가정(8,000 테넌트 = 1,600 rps) 하에 6개 부하 수준(100–1,599 rps)에 대해 회귀한 결과, p99 예측 모형은 $p99 \approx 2.353 \cdot rps - 47.5$ ($R^2 = 0.996$, $n = 6$)으로 도출되었다. 결과적으로 컨테이너는 약 41 테넌트의 hard wall을 보인 반면 Wasm 풀은 점진적 성능 저하(graceful degradation)를 보여, 격리 요구가 강한 계층에서 Wasm이 유의하게 우세하였다.

다만 컨테이너의 약 41 Pod 한계는 컨테이너 런타임 자체의 절대적 한계를 의미하지 않는다. 본 연구에서는 쿠버네티스와 Docker가 권장하는 최소 리소스 설정(128 Mi request)을 적용하였으며, request 값을 낮추면 더 많은 Pod를 수용할 수 있다. 따라서 본 결과는 실무 권장 설정 기준에서의 상대 비교로 해석해야 한다.

4.4 격리의 경제성

per-tenant 격리 비용을 월 단위로 환산한 결과, 테넌트 규모가 커질수록 격차가 확대되었다(그림 4). 200 테넌트에서 5.3×, 1,000 테넌트에서 25.7×, 2,500 테넌트에서 최대 64.3×(\$4.69 대 \$0.073)로 Wasm이 저렴하였다. 컨테이너는 테넌트당 노드 비용이 선형으로 증가하는 반면 Wasm 공유 풀은 고정 비용을 분할 상환하기 때문이며, 이 모형은 Pod 밀도 실측으로 검증되었다.

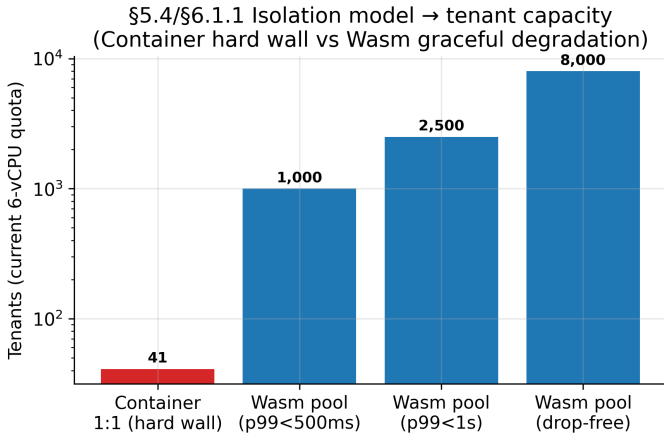


그림 3: 격리 모델별 수용 테넌트 수(로그 스케일). 컨테이너 1:1은 약 41에서 한계, Wasm 풀은 8,000 이상.

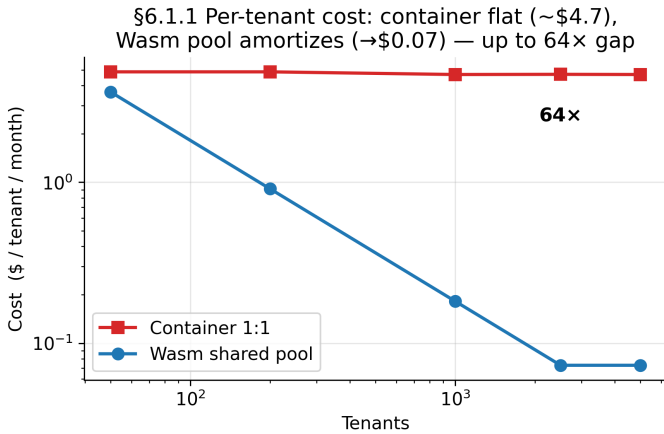


그림 4: 테넌트당 월 비용(log-log) — 컨테이너 약 \$4.7로 평탄, Wasm은 \$0.07까지 감소하여 최대 64× 격차.

4.5 Edge 계층, scale-to-zero 및 운영

Edge 계층에서는 Pod Ready 시간이 Wasm 1,085 ms(SD 769) 대 컨테이너 2,161 ms(SD 391)로 Wasm이 약 2× 우세하였고 ($d = -1.76, n = 10$), 이미지 Cold Pull은 Wasm 1,078 ms(SD 166) 대 컨테이너 1,307 ms(SD 7)로 본 연구 최대 효과크기 ($d = -7.19, n = 10$, Bonferroni 유의)를 보였다. 다만 Cold Pull의 절대 격차는 1.21×에 그치며, 이처럼 큰 효과크기는 두 집단의 분산이 작은 데 기인하므로 표준화 효과크기를 실용적 격차의 크기로 해석해서는 안 된다. 또한 arm64로 빌드한 컨테이너 이미지가 amd64 노드에서 `exec format error`로 실패한 반면 동일 Wasm 모듈은 무수정으로 작동하여 cross-arch 이식성을 실증하였다. scale-from-zero cold-start에서는 Wasm 269 ms(SD 46) 대 컨테이너 2,979 ms(SD 577)로 약 11× 우세하였으나, idle 상태 실측 메모리는 컨테이너(2 Mi)가 Wasm(24 Mi)보다 작아 “idle 메모리 절감” 가설은 기각되었다. 운영 측면에서 MTTR과 rolling update는 Wasm이 1.3–1.4× 우세(무중단 5/5)하였으나, 버전 롤백과 설정 교체는 layer 캐시를 활용하는 컨테이너가 8–10× 우세하여 운영상의 우위는 작업 유형에 따라 갈리는 것으로 나타났다.

표 2: 계층별 권장 런타임과 근거

계층	권장 런타임	근거
Edge	Wasm	Cold Pull $d = -7.19$, 이식성
Transform	Wasm	격리 41→8,000, 64× 저렴
Ingestion	Container	약 19–22×, 24–28K rps

4.6 워크로드 복잡도와 IoT 종단 검증

워크로드 복잡도를 L0(단순)부터 L4(SHA-256+gzip)까지 변화시킨 sweep에서, L0에서는 컨테이너가 1.5× 우세하였으나 L1 정규식에서는 유의한 차이가 없었고($p = 0.98$, 검정력 제한) L4에서는 p50 기준 Wasm이 30% 우세(평균 기반 $d = -0.05$, negligible)하였다. 따라서 “복잡한 워크로드가 Wasm에 불리하다”는 가설은 기각되었다. 한편 IoT Hub에서 Event Hubs로 이어지는 MQTT 종단 경로의 안정성은 $n = 30$ 에서 187.5 ± 89.2 ms로 측정되었으며, 로봇공장 630건, 스마트팜 180건, 차량 1,800건 시나리오 모두에서 손실이 발생하지 않았다.

4.7 계층별 종합

계층별 측정 결과를 종합하면 표 2와 같다. Ingestion 계층은 처리량 우위로 컨테이너가, Transform과 Edge 계층은 per-tenant 격리, 비용, 이식성 우위로 Wasm이 적합한 것으로 나타났다. 즉 본 연구의 결과는 단일 런타임의 우열이 아니라 파이프라인 계층별 요구사항에 따른 런타임 할당의 필요성을 실측으로 뒷받침한다.

4.8 Threats to Validity

본 연구의 결과는 다음 네 가지 타당성 위협을 전제로 해석되어야 한다. 첫째, 측정은 6 vCPU 규모의 AKS free-tier 클러스터에서 수행되었으므로 41 Pod plateau나 8,000 테넌트 ceiling과 같은 절대 수치는 production-scale 환경과 다를 수 있다. 다만 두 런타임에 동일 쿼터를 적용한 상대 비교의 방향성은 유지된다고 본다. 둘째, 워크로드가 단일 Go IoT 로그 처리 앱으로 한정되어 다른 언어나 로직으로의 일반화에는 추가 검증이 필요하며, 특히 Wasm 생태계에서 널리 쓰이는 Rust 기반 워크로드에서는 상이한 결과가 나타날 수 있다. 셋째, Wasm 런타임이 SpinKube/Wasmtime으로 한정되어 WasmEdge 등 다른 Wasm 런타임의 특성은 반영하지 못하였다. 넷째, 컨테이너 측 측정이 쿠버네티스 권장 리소스 요청(128 Mi)을 기준으로 하므로, 더 낮은 request나 다른 리소스 정책에서는 테넌트 밀도와 비용 결과가 달라질 수 있다. 다섯째, 처리량 비교는 Wasm 5-Pod 풀과 컨테이너 배치 간 유효 Pod·CPU 할당이 완전히 동일하지 않을 수 있어, 처리량 격차의 일부는 자원 배분 차이를 반영할 가능성이 있다. 여섯째, instantiation은 shim journal 타임스탬프에서 추출되어 컨테이너 baseline의 측정 경로와 시계 기준이 달라 측정 도구상의 비교 한계가 존재한다.

5 결론

본 연구에서는 동일한 Go 워크로드를 SpinKube 기반 WebAssembly 모듈과 scratch 컨테이너로 구현하여, Azure Kubernetes Service(AKS 1.34) 위에서 IoT 로그 파이프라인의 Edge·Transform·Ingestion 세 계층에 걸쳐 다수의 측정 세션으로 두 런타임을 비교하였다. 그 결과 Fermiyon이 주장한

인스턴스화 0.5 ms 수준의 런타임 성능은 실측 0.436 ms로 유의하게 다르지 않아($p=0.71$, $n = 10$) 런타임 수준의 주장을 기각하지 못하였으나(검정력 제한), 전체 요청 경로 1.55 ms 가운데 런타임이 차지하는 비중은 28%에 불과하고 나머지 72%가 쿠버네티스, shim, SDK, HTTP 계층에서 더해지는 것으로 나타났다. 즉 벤더가 제시한 정량 이점은 운영 스택을 통과하면서 크게 희석된다.

계층별로는 상반된 양상이 관찰되었다. Ingestion 계층에서는 컨테이너가 24–28K rps를 일관되게 유지하여 Wasm(약 1,264 rps)의 약 19–22×로 유의하게 우세하였다. 반면 Transform 계층에서 테넌트당 격리를 강제하면 컨테이너의 메모리 요청 예약이 노드를 채워 약 41 Pod에서 한계에 이르는 데 반해, 고정 Wasm 공유 풀은 8,000 테넌트까지 무손실로 수용하였으며 테넌트당 비용은 최대 64× 저렴한 것으로 나타났다. 또한 scale-to-zero cold-start에서 Wasm이 11× 우세하고 cross-arch 이식성에서도 우위를 보였으나, 버전 롤백과 설정 교체에서는 컨테이너가 8–10× 우세하여 우위가 요구사항 축에 따라 갈리는 것으로 나타났다.

본 연구의 핵심 결론은 단일 런타임 선택이 아니라 계층별 할당이다. 고동시성 수집을 담당하는 Ingestion 계층에는 컨테이너를 배치하고, per-tenant 격리와 이식성이 요구되는 Transform 및 Edge 계층에는 Wasm을 배치하는 것이 타당하다. 이는 마케팅 수치가 아니라 실험 환경 측정에 근거한다. 다만 본 측정은 6 vCPU 규모의 free-tier 클러스터에서 수행되어 41 Pod plateau와 8,000 테넌트 ceiling은 쿼터 상대적 하한값이라는 한계를 가지며, 컨테이너의 메모리 요청 가정과 단일 워크로드, 단일 클라우드 환경 또한 일반화의 제약이 된다. 또한 본 연구의 계층별 분석은 Edge, Transform, Ingestion 계층에 한정되며, Storage 계층은 스토리지 엔진의 영향이 지배적이어서 후속 연구로 남겨둔다. 향후 연구로는 production-scale 쿼터에서의 재측정, 멀티 워크로드 및 멀티 런타임으로의 확장, Storage 계층을 포함한 종단 파이프라인 통합, 그리고 stateful 처리를 포함한 후속 검증이 필요하다.

References

- [1] S. Shillaker and P. Pietzuch, “Faasm: Lightweight Isolation for Efficient Stateful Serverless Computing,” in *USENIX ATC*, 2020, pp. 419–433.
- [2] A. Bacchilega et al., “Lumos: WebAssembly across the Edge–Cloud Continuum,” arXiv:2510.05118, 2025.
- [3] M. Alzayat, J. Mace, P. Druschel, and D. Garg, “Groundhog: Efficient Request Isolation in FaaS,” in *EuroSys*, 2023, pp. 398–415.
- [4] J. A. Wiegatz, “Comparing Security and Efficiency of WebAssembly and Linux Containers in Kubernetes,” arXiv:2411.03344, 2024.
- [5] Fermyon Technologies, “Spin: Serverless WebAssembly at the Speed of Light — Millisecond Cold Starts,” Fermyon Developer Documentation, 2024. [Online]. Available: <https://developer.fermyon.com/spin>
- [6] Envoy Proxy, “WebAssembly (Wasm) HTTP Filter,” Envoy Documentation, 2023.

- [7] Fluent Bit, “WASM Filter Plugins,” Fluent Bit Documentation, 2023.